

xTest接入智能交换芯片自适应测试系统设计报告

基于 /Users/zhao/Desktop/my_projects/xtest 本地代码结构、知识库、AI用例生成、TCL自动化与任务中心的落地方案

2026-06-30 18:06 · AI领航员·DC技术研究助手

xTest 接入智能交换芯片多代系自适应自动化测试系统详细设计报告 V2.0

报告日期: 2026-06-30

分析对象: /Users/zhao/Desktop/my_projects/xtest

目标系统: xTest 1.0.0, Node.js + Express + MySQL 测试管理平台

目标文档: 《智能交换芯片多代系自适应自动化测试系统详细设计文档 V2.0》

输出形态: 精美 HTML 邮件正文 + PDF 附件

署名: AI领航员·DC技术研究助手

0. 执行摘要

本次分析结论很明确: 不要在 xTest 之外重建一套新平台, 而应把《智能交换芯片多代系自适应自动化测试系统详细设计文档 V2.0》拆解为 xTest 现有知识库、AI 用例生成、TCL 自动化生成、任务中心、反思管道、脚本执行与邮件服务的增量增强。

xTest 当前已经具备落地该系统的关键底座:

- 知识库/VFS:** routes/knowledge.js、services/vfsService.js、/services/fileParserService, 支持 doc/docx/xls/xlsx/pdf/pptx/txt/md/drawio/vsd 上传、解析、预览与分块。
- 网页资料入库:** /services/webCrawlerService 已有 URL 校验、内网地址阻断、HTML 主体抽取、Markdown 转换能力, 可用于标准文档、芯片勘误、厂商网页资料入库。
- Office/PDF 预览:** routes/knowledge.js 里有 checkLibreOffice 与 convertToPdfViaLibreOffice, 候选路径明确包含 /Applications/LibreOffice.app/Contents/MacOS/soffice。
- 知识分块与向量检索:** services/chunkingService.js + services/embeddingAdapter.js, 当前以 MySQL JSON 存储 embedding, 并以内存余弦相似度检索; EMBEDDING_BACKEND=chroma 的分支存在, 但当前会抛出 ChromaDB后端尚未实现。
- AI 测试用例生成:** services/caseGeneratorService.js 已实现 Map-Reduce 式分块生成、任务进度、临时用例落库、查重、用户确认。
- TCL 自动化生成:** services/tclGenerationService.js + routes/tclGeneration.js 已实现 RAG 检索 CLI/TCL 规范、三段式 TCL 模板、静态校验、动态执行、失败自修复、经验回写。

- **TCL 学习**: `services/tclLearningService.js` 可从历史 TCL 脚本中提取 CLI 命令模式与 TCL 编码规范, 并回写 `cli / tcl_convention` 知识块。
- **5 维测试点扩展**: `services/testPointExpansionService.js` 已定义 `functional`、`boundary`、`exception`、`combination`、`state` 五个维度。
- **统一 AI 任务中心**: `services/unifiedTaskService.js` + `routes/aiTasks.js` 已有 `ai_unified_tasks` 状态机。
- **Web UI 入口**: `public/js/level1-points-list.js` 已存在 `generateTCL` 与 `expandPoints` 入口;
`public/css/knowledge-library.css` 已形成完整知识库 UI 风格。

推荐路线: 第一阶段复用 xTest MySQL-only RAG 底座, 把 `chip_version`、SVD 寄存器字典、SDK API 映射、Bug-RAG、执行环境矩阵、证据链和迁移中心加进去; 第二阶段再考虑 ChromaDB、tree-sitter/clangd、远程 runner 集群。

1. xTest 当前系统能力盘点

1.1 工程与运行栈

维度	当前状态	关键文件
项目路径	<code>/Users/zhao/Desktop/my_projects/xtest</code>	用户指定路径
项目名	<code>xtest</code>	<code>package.json</code>
版本	<code>1.0.0</code>	<code>package.json</code>
主入口	<code>server.js</code>	<code>package.json</code> / <code>server.js</code>
后端框架	Express	<code>server.js</code>
数据库	MySQL	<code>db.js</code> 、 <code>migrations/</code>
前端	原生 HTML/CSS/JS	<code>public/</code> 、根目录 <code>index.html</code>
AI 调用	axios + AI 配置 + <code>aiCallWrapper</code>	<code>services/aiService.js</code> 、 <code>services/aiCallWrapper.js</code>
邮件	nodemailer	<code>services/emailService.js</code>
自动迁移	启动内建迁移 + SQL migrations	<code>server.js</code> 、 <code>migrations/</code>

`server.js` 已挂载与本次落地最相关的路由:

```
app.use('/api/knowledge', require('./routes/knowledge'));
app.use('/api/tcl-generation', require('./routes/tclGeneration'));
app.use('/api/ai-generation', require('./routes/aiGeneration'));
app.use('/api/ai-tasks', require('./routes/aiTasks'));
app.use('/api/temp-cases', require('./routes/tempCases'));
app.use('/api', require('./routes/scripts'));
```

这说明 xTest 已经不是简单测试用例管理系统, 而是一个具备“知识资产 → AI 生成 → 审核落库 → 自动化脚本”的测试工程平台。

1.2 知识库能力

`routes/knowledge.js` 提供 VFS 知识库入口, 核心能力包括:

1. **全局/模块/用例库文件树**: `/api/knowledge/global-tree`、`/api/knowledge/tree/:moduleId`、`/api/knowledge/files/:moduleId`、`/api/knowledge/library-files/:libraryId`。

- 2. 上传与批量上传： `/api/knowledge/upload` 、 `/api/knowledge/upload-batch` 。
- 3. 文件类别标签：当前已有 `design` 、 `cli` 、 `methodology` 、 `tcl_convention` 、 `environment` 、 `test_plan` 、 `tcl_script` 。
- 4. 解析服务： `/services/fileParserService` 支持 Word、Excel、PDF、PPT、TXT、Markdown 等格式。
- 5. 结构感知切分： `fileParserService.chunkContent` 会识别标题、代码块、表格、列表、段落、句子边界。
- 6. Office 预览： `checkLibreOffice` 与 `convertToPdfViaLibreOffice` 支持通过 LibreOffice 转 PDF，候选路径包括 `/Applications/LibreOffice.app/Contents/MacOS/soffice` 。
- 7. 网页爬取： `/services/webCrawlerService` 提供 `validateUrl` ，阻止 `127.0.0.1` 、 `localhost` 、 `10.` 、 `172.16/12` 、 `192.168.` 、 `169.254.*` 、 `metadata.google.internal` 等内网/元数据地址。

评价：这层可以直接承接《V2.0》的 Data & Knowledge Layer，但需要扩展芯片代系、SVD、SDK API、Bug-RAG 等类型。

1.3 AI 用例生成能力

`services/caseGeneratorService.js` 的主链路：

```
module_knowledge_files
→ ai_material_chunks
→ ai_case_generation_tasks
→ executeMapPhase 分块生成
→ temp_level1_points / temp_test_cases
→ 查重与用户确认
→ level1_points / test_cases
```

关键特征：

- 支持 `selectedFiles` ，用户可指定知识库文件生成。
- 通过 `CHUNK_API_CONCURRENCY` 控制分块 AI 调用并发。
- 通过 `callAIStreamWithRetry` 做 AI 调用与重试。
- 通过 `dedupService` 做文本相似度查重。
- 生成结果先进入临时表，不直接污染正式用例库。

差距：当前生成逻辑还没有把 `chip_version` 作为全局一等公民，也缺少寄存器、SDK API、历史 Bug、执行日志的结构化证据链。

1.4 TCL 自动化能力

`routes/tclGeneration.js` 提供：

API	功能
POST <code>/api/tcl-generation/generate</code>	为一级测试点生成 TCL 脚本
GET <code>/api/tcl-generation/task/:taskId</code>	查询生成任务
POST <code>/api/tcl-generation/expand-points</code>	对测试点做 5 维扩展

任务 ID 模板：

```
TCL-${new Date().toISOString().slice(0, 10).replace(/-/g, '')}-${uuidv4().slice(0, 8).toUpperCase()}
```

`services/tclGenerationService.js` 当前流程：

```
创建 tcl_generation_tasks
→ 查询 level1_points
→ _assembleKnowledge 检索 cli / tcl_convention
→ _generateTCL 生成 Setup/Test Steps/Teardown 三段式脚本
→ _staticValidation 校验 proc setup / proc test_steps / proc teardown / catch / 括号匹配
→ _dynamicExecution 调 scriptRunnerClient
→ _selfRepairLoop 根据 stderr/stdout 修复
→ _writeRepairExperience 回写 execution_experience
→ _saveAndAssociate 保存脚本并关联
```

services/scriptRunnerClient.js 通过 SCRIPT_RUNNER_URL 调外部执行器；未配置时返回：ScriptRunner Agent未配置，执行状态为 skipped。

评价：这已经是《V2.0》里 Scripter/Critic/Executor/Reflection 的最小闭环，只需补强芯片代系、SVD/SDK 证据、多环境执行和人工审核。

1.5 RAG 与 Embedding 现状

services/embeddingAdapter.js 当前非常关键：

- 默认 EMBEDDING_BACKEND=mysql。
- 如果设置为 chroma，会抛出 ChromaDB后端尚未实现。
- 检索支持过滤：fileCategory、moduleId、libraryId。
- 向量存放在 ai_material_chunks.embedding，检索时用内存余弦相似度排序。

这符合 xTest 既有“极简实用主义三不原则”：

不采用	当前替代
不引入 Redis	MySQL 状态机 + p-queue
不引入向量数据库	MySQL JSON embedding + 内存余弦相似度
不使用复杂 RAG 框架	MySQL 分块 + Map-Reduce + 自研检索

建议：短期继续 MySQL RAG；中期再把 ChromaDB 做成可选后端，而不是硬依赖。

2. 《V2.0》能力到 xTest 的映射

《V2.0》能力	xTest 现有承接点	复用度	缺口	设计建议
多代芯片上下文	modules、case_libraries、module_knowledge_files	50%	无统一 chip_version	新增 chip_versions 与上下文解析服务
SVD 寄存器字典	知识库上传/解析	30%	无 XML/SVD 专用解析器	新增 svdParserService 与寄存器表
SDK AST/API 映射	知识库代码文件	20%	无 API symbol 索引	新增 sdkAstService，先正则后 AST
Doc RAG	ai_material_chunks + embeddingAdapter	75%	分类粗，证据弱	扩展 file_category 与 metadata
Bug-RAG	execution_experience、tcl_learning_results	45%	缺人工审核候选池	新增 bug_rag_entries

《V2.0》能力	xTest 现有承接点	复用度	缺口	设计建议
Planner Agent	<code>caseGeneratorService</code> 、 <code>unifiedTaskService</code>	55%	无多阶段 DAG	新增 <code>adaptiveWorkflowService</code>
Search/RAG Agent	<code>embeddingAdapter.search</code>	65%	检索类别少	新增 hybridSearch / evidence capture
Scripter Agent	<code>tclGenerationService._generateTCL</code>	70%	缺 SVD/SDK 约束	扩展 prompt 与 RAG 类别
Critic/Lint Agent	<code>_staticValidation</code> 、 <code>reflectionPipeline</code>	55%	规则不覆盖芯片/寄存器/SDK	新增规则集
Executor Agent	<code>scriptRunnerClient</code>	40%	环境模型弱	新增执行环境表与 runner adapter
Reflection 回灌	<code>_writeRepairExperience</code>	55%	缺审核 UI	新增候选池与审核流
Web UI	<code>knowledge-library.css</code> 、 <code>level1-points-list.js</code>	60%	缺迁移中心/证据视图	增量新增页面和抽屉

最重要的工程原则：

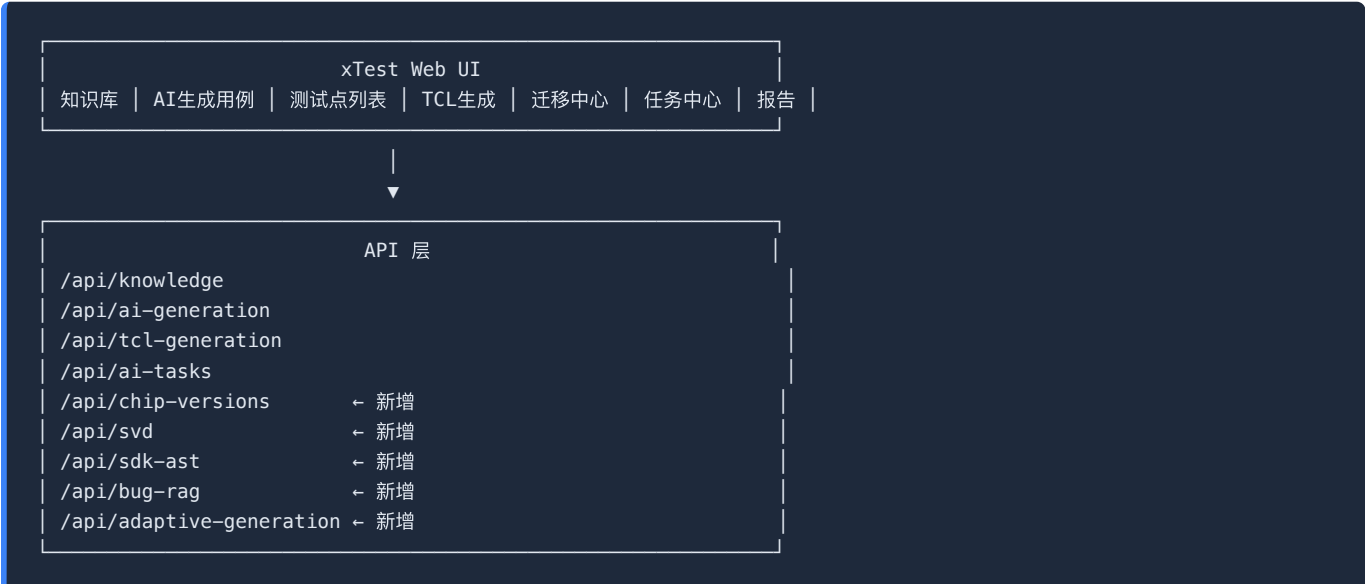
`chip_version` 必须作为全局一等公民；事实层与方法层分离；Agent 输出必须带证据链；生成脚本执行前必须经过 Critic/Lint；失败结果必须进入 Reflection/Bug-RAG 候选并经人工审核。

落到 xTest 后应转化为四条硬约束：

- 1. 所有知识块都必须知道自己属于哪一代芯片，或显式标记为跨代通用。
- 2. 所有生成结果必须记录引用了哪些 `chunk_id`、寄存器字段、SDK API、历史 Bug。
- 3. 所有脚本必须先静态检查，再按环境动态执行；执行失败不能被丢弃。
- 4. 所有失败经验先进入候选池，经人工审核后才能进入正式 Bug-RAG。

3. 目标架构设计

3.1 xTest 内生增强架构





3.2 事实层与方法层分离

层次	含义	xTest 承接方式	示例
事实层	随芯片代系变化的数据	chip_versions、chip_registers、sdk_api_symbols、module_knowledge_files.file_category	Gen1/Gen2/Gen3 地址、字段、SDK API 差异
方法层	跨代稳定的测试逻辑	level1_points、test_cases、methodology、tcl_convention	PFC deadlock recovery 测试逻辑
生成层	基于事实层实例化方法层	adaptive_case_generation、tcl_generation_tasks	将 PFC 测试模板映射到 Gen3 寄存器
反馈层	执行失败、修复、经验	execution_experience、bug_rag_entries	POLL_TIMEOUT、LEGACY_ADDR_HARDCODED

4. 数据库改造设计

4.1 芯片代系表

```
CREATE TABLE chip_versions (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  chip_key VARCHAR(100) NOT NULL UNIQUE COMMENT '如 Switch_Gen3_RevB',  
  chip_name VARCHAR(200) NOT NULL,  
  generation VARCHAR(50) NOT NULL COMMENT 'Gen1/Gen2/Gen3',  
  revision VARCHAR(50) DEFAULT NULL COMMENT 'RevA/RevB',  
  parent_chip_key VARCHAR(100) DEFAULT NULL COMMENT '继承来源, 如 Switch_Gen2',  
  status ENUM('planning','active','deprecated') DEFAULT 'active',  
  description TEXT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  INDEX idx_generation (generation),
```

```
INDEX idx_parent (parent_chip_key)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

初始数据建议：

```
INSERT INTO chip_versions (chip_key, chip_name, generation, revision, parent_chip_key) VALUES
('Switch_Gen1', 'Switch Gen1', 'Gen1', NULL, NULL),
('Switch_Gen2', 'Switch Gen2', 'Gen2', NULL, 'Switch_Gen1'),
('Switch_Gen3', 'Switch Gen3', 'Gen3', NULL, 'Switch_Gen2'),
('Switch_Gen3_RevB', 'Switch Gen3 RevB', 'Gen3', 'RevB', 'Switch_Gen3');
```

4.2 绑定 `chip_version`

最小侵入字段：

```
ALTER TABLE module_knowledge_files
ADD COLUMN chip_version_id INT NULL COMMENT '适用芯片代系，NULL表示通用';

ALTER TABLE ai_material_chunks
ADD COLUMN chip_version_id INT NULL COMMENT '冗余字段，便于RAG过滤';

ALTER TABLE level1_points
ADD COLUMN chip_version_id INT NULL COMMENT '测试点适用芯片代系';

ALTER TABLE test_cases
ADD COLUMN chip_version_id INT NULL COMMENT '用例适用芯片代系';

ALTER TABLE tcl_generation_tasks
ADD COLUMN chip_version_id INT NULL COMMENT '生成目标芯片代系';
```

4.3 SVD 寄存器表

```
CREATE TABLE chip_registers (
  id INT AUTO_INCREMENT PRIMARY KEY,
  chip_version_id INT NOT NULL,
  peripheral_name VARCHAR(128) NOT NULL,
  register_name VARCHAR(128) NOT NULL,
  address_offset BIGINT NOT NULL,
  absolute_address BIGINT NULL,
  access_type VARCHAR(32),
  reset_value VARCHAR(64),
  description TEXT,
  source_file_id INT NULL,
  source_hash VARCHAR(64),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uk_chip_reg (chip_version_id, peripheral_name, register_name),
  INDEX idx_addr (chip_version_id, absolute_address),
  INDEX idx_reg_name (register_name)
);

CREATE TABLE chip_register_fields (
  id INT AUTO_INCREMENT PRIMARY KEY,
  register_id INT NOT NULL,
  field_name VARCHAR(128) NOT NULL,
  bit_offset INT NOT NULL,
  bit_width INT NOT NULL,
  access_type VARCHAR(32),
  enum_values JSON,
  description TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```

    UNIQUE KEY uk_reg_field (register_id, field_name),
    INDEX idx_field_name (field_name)
);

```

示例实体：`PFC_DEADLOCK_TIMER`、`PFC_DEADLOCK_TIMER.THRESHOLD`、`PFC_RECOVERY_CTRL`、`PFC_RECOVERY_STATUS`、`FSM_STATE`、`RECOVERED`。

4.4 SDK API 索引表

```

CREATE TABLE sdk_api_symbols (
  id INT AUTO_INCREMENT PRIMARY KEY,
  chip_version_id INT NULL,
  symbol_name VARCHAR(200) NOT NULL,
  symbol_type ENUM('function','macro','struct','enum','field') DEFAULT 'function',
  signature TEXT,
  header_file VARCHAR(500),
  source_file VARCHAR(500),
  module_name VARCHAR(100),
  description TEXT,
  source_file_id INT NULL,
  metadata JSON,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_symbol (symbol_name),
  INDEX idx_chip_symbol (chip_version_id, symbol_name)
);

```

第一阶段只做正则/ctags 风格解析；第二阶段再引入 tree-sitter 或 clangd。

4.5 证据链表

```

CREATE TABLE evidence_links (
  id INT AUTO_INCREMENT PRIMARY KEY,
  target_type VARCHAR(50) NOT NULL COMMENT 'level1_point/test_case/tcl_task/script/bug_rag',
  target_id VARCHAR(100) NOT NULL,
  evidence_type VARCHAR(50) NOT NULL COMMENT 'chunk/register/field/sdk_api/bug/exec_log',
  evidence_id VARCHAR(100) NOT NULL,
  relevance DECIMAL(6,4) DEFAULT NULL,
  quote TEXT,
  metadata JSON,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_target (target_type, target_id),
  INDEX idx_evidence (evidence_type, evidence_id)
);

```

4.6 Bug-RAG 候选池

```

CREATE TABLE bug_rag_entries (
  id INT AUTO_INCREMENT PRIMARY KEY,
  bug_key VARCHAR(100) UNIQUE,
  chip_version_id INT NULL,
  module_id INT NULL,
  symptom TEXT NOT NULL,
  root_cause TEXT,
  fix_strategy TEXT,
  wrong_pattern TEXT,
  correct_pattern TEXT,
  error_code VARCHAR(100),
  source_task_id VARCHAR(100),
  source_execution_id VARCHAR(100),

```



```

status ENUM('candidate','approved','rejected','archived') DEFAULT 'candidate',
reviewer_id INT NULL,
reviewed_at TIMESTAMP NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
INDEX idx_status (status),
INDEX idx_chip_module (chip_version_id, module_id),
FULLTEXT KEY ft_bug_text (symptom, root_cause, fix_strategy)
);

```

示例错误模式： `FIELD_VALUE_OUT_OF_RANGE`、`LEGACY_ADDR_HARDCODED`、`POLL_TIMEOUT`、`BUG-PFC-2026-0017`、`pfc_recovery_fsm_clear_timeout`。

4.7 执行环境矩阵

```

CREATE TABLE execution_environments (
  id INT AUTO_INCREMENT PRIMARY KEY,
  env_key VARCHAR(100) NOT NULL UNIQUE,
  env_name VARCHAR(200) NOT NULL,
  env_type ENUM('sim','fpga','silicon','sdk_unit','other') NOT NULL,
  runner_url VARCHAR(500),
  default_timeout_sec INT DEFAULT 300,
  status ENUM('active','inactive','maintenance') DEFAULT 'active',
  config JSON,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

CREATE TABLE script_execution_runs (
  id INT AUTO_INCREMENT PRIMARY KEY,
  task_id VARCHAR(100),
  script_id INT NULL,
  env_id INT NULL,
  chip_version_id INT NULL,
  status ENUM('pending','running','passed','failed','timeout','skipped','agent_unreachable') DEFAULT 'pending',
  exit_code INT NULL,
  stdout LONGTEXT,
  stderr LONGTEXT,
  duration_ms INT NULL,
  metadata JSON,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  completed_at TIMESTAMP NULL
);

```

建议初始环境： `xcelium_sim`、`vcs_sim`、`fpga_lab`、`silicon_bringup`、`sdk_unit_test`。

5. 后端服务改造设计

5.1 ChipContextService

新增文件：

```

/services/chipContextService.js
/routes/chipVersions.js

```

核心方法：

```
async resolveContext({ moduleId, libraryId, levelPointId, testCaseId, chipVersionId })
async getInheritanceChain(chipVersionId)
async isCompatible(sourceChipVersionId, targetChipVersionId)
async buildPromptContext(chipVersionId)
```

Prompt 上下文示例：

```
{
  "chip_version": "Switch_Gen3_RevB",
  "generation": "Gen3",
  "parent": "Switch_Gen3",
  "inheritance": ["Switch_Gen1", "Switch_Gen2", "Switch_Gen3", "Switch_Gen3_RevB"],
  "compatibility_policy": "specific_first_then_parent_then_common"
}
```

5.2 SVDParserService

新增：

```
/services/svdParserService.js
/routes/svd.js
```

流程：

```
知识库上传 SVD XML
→ fileParserService 识别 xml/svd
→ svdParserService.parse(fileId, chipVersionId)
→ chip_registers / chip_register_fields 落库
→ 为 register/field 生成 Markdown 摘要块
→ embeddingAdapter.upsertKnowledgeChunks(fileCategory='register_map')
```

5.3 SDKAstService

新增：

```
/services/sdkAstService.js
/routes/sdkAst.js
```

阶段策略：

阶段	方式	目标
P1	正则 + ctags 风格解析	抽取函数、宏、struct、enum、头文件路径
P2	tree-sitter/clangd	获取参数类型、调用关系、注释、跨文件索引

输出 `sdk_api_symbols`，同时生成 `file_category='sdk_api'` 的知识块。

5.4 扩展 EmbeddingAdapter

新增 `hybridSearch`：

```
async hybridSearch(queryText, topK, filters = {}) {
  const categories = filters.categories || ['design'];
  const all = [];
```

```

for (const fileCategory of categories) {
  const rows = await this.search(queryText, topK, { ...filters, fileCategory });
  all.push(...rows);
}
return rerankAndDedup(all).slice(0, topK);
}

```

新增知识类别：

```

register_map
register_field
sdk_api
errata
bug_rag
execution_experience
migration_note

```

短期上线注意：不要把 `EMBEDDING_BACKEND` 设成 `chroma`，因为当前会触发 `ChromaDB后端尚未实现`。

5.5 修复 `/services/fileParserService` 中 TCL 学习后处理位置

当前 `parseAndChunk` 在成功保存 chunks 后已经 `return`，后面的 TCL 学习后处理代码在逻辑上可能不可达。建议将：

```

if file_category === 'tcl_script' → tclLearningService.analyzeFromFile(...)

```

移动到保存 chunks 与更新 `parse_status` 之后、最终 `return` 之前。

这是一个必须先修的工程点，否则上传历史 TCL 脚本后可能只完成分块，不会自动提炼 CLI/TCL 规范。

5.6 `AdaptiveWorkflowService`

新增：

```

/services/adaptiveWorkflowService.js
/routes/adaptiveGeneration.js

```

职责：把 Planner/Search/Scripter/Critic/Executor/Reflection 串成一个可追踪任务。

```

createAdaptiveTask
→ resolve chip context
→ retrieve design/register/sdk/bug evidence
→ generate level1/test_cases
→ generate TCL/SDK script
→ lint/critic
→ execute on selected env
→ collect evidence
→ write reflection candidate

```

5.7 扩展 `TCLGenerationService`

当前 `_assembleKnowledge` 只检索 `cli` 和 `tcl_convention`。建议改为：

```

const ragContext = await embeddingAdapter.hybridSearch(level1PointName, 5, {
  moduleId,
  chipVersionId,
  categories: [
    'cli',

```

```
    'tcl_convention',
    'register_map',
    'register_field',
    'sdk_api',
    'bug_rag',
    'execution_experience',
    'environment'
  ]
};
```

Prompt 必须增加：

- 当前芯片代系。
- 禁止硬编码旧代地址。
- 寄存器字段必须来自 SVD/寄存器字典证据。
- CLI 命令必须来自 `cli` 证据。
- 已知失败模式必须规避。
- 输出脚本必须带注释形式的证据摘要，例如：

```
# Evidence:
# - Register: PFC_DEADLOCK_TIMER.THRESHOLD (Switch_Gen3_RevB)
# - Bug-RAG: BUG-PFC-2026-0017 / POLL_TIMEOUT
```

6. API 设计

6.1 芯片代系 API

API	方法	功能
<code>/api/chip-versions</code>	GET	查询芯片代系列表
<code>/api/chip-versions</code>	POST	新建芯片代系
<code>/api/chip-versions/:id</code>	PUT	更新芯片代系
<code>/api/chip-versions/:id/inheritance</code>	GET	查询继承链
<code>/api/chip-versions/:id/bindings</code>	GET	查询该代系关联的知识/用例/脚本

6.2 SVD API

API	方法	功能
<code>/api/svd/parse</code>	POST	解析指定知识库 SVD 文件
<code>/api/svd/registers</code>	GET	查询寄存器
<code>/api/svd/registers/:id/fields</code>	GET	查询字段
<code>/api/svd/diff</code>	POST	比较两代芯片寄存器差异

6.3 SDK API 索引

API	方法	功能
/api/sdk-ast/index	POST	索引 SDK 源码/头文件
/api/sdk-ast/symbols	GET	查询符号
/api/sdk-ast/diff	POST	比较两代 SDK API 差异

6.4 自适应生成 API

API	方法	功能
/api/adaptive-generation/create	POST	创建多代自适应生成任务
/api/adaptive-generation/task/:taskId	GET	查询任务
/api/adaptive-generation/evidence/:targetType/:targetId	GET	查询证据链

请求示例：

```
{
  "moduleId": 12,
  "libraryId": 3,
  "chipVersionId": 4,
  "selectedFiles": [101, 102, 103],
  "target": "tcl_and_cases",
  "executionEnv": "xcelium_sim",
  "enableBugRag": true,
  "enableReflection": true
}
```

7. 前端 UI 设计

7.1 知识库上传增强

在现有知识库上传对话框中增加：

- 芯片代系：通用 / Switch_Gen1 / Switch_Gen2 / Switch_Gen3 / Switch_Gen3_RevB
- 文件类型：扩展为 svd 、 sdk_source 、 sdk_header 、 errata 、 bug_rag 、 migration_note
- 解析策略：普通文本 / 结构感知 / SVD 寄存器解析 / SDK API 索引
- 上传后动作：仅入库 / 入库并生成 embedding / 入库并解析结构化资产 / 入库并触发 TCL 学习

UI 草图：

上传知识文件

文件： [选择文件]

芯片代系： [Switch_Gen3_RevB ▼]

文件类型： [SVD寄存器描述 ▼]

解析策略： [SVD结构化解析 + RAG入库 ▼]

上传后动作：

☒ 生成知识块 ☒ 生成Embedding ☒ 建立证据索引

[取消]

[上传并解析]

7.2 测试点列表增强

`public/js/level1-points-list.js` 已有 `generateTCL` 与 `expandPoints`。建议扩展为：

- 单点按钮：`生成 TCL`、`生成 SDK 单测`、`查看证据`、`迁移到新芯片`
- 批量按钮：`5维扩展`、`批量生成自动化脚本`、`批量执行`、`批量进入迁移评估`
- 列表字段：芯片代系、证据数量、自动化脚本数量、最近执行状态、Bug-RAG 命中数

7.3 迁移中心

新增页面：

```
public/chip-migration.html
public/js/chip-migration.js
public/css/chip-migration.css
```

页面核心区块：

1. 源芯片与目标芯片选择。
2. SVD 差异概览。
3. SDK API 差异概览。
4. 可继承测试点列表。
5. 需重生成本脚本列表。
6. 高风险 Bug-RAG 命中列表。
7. 一键创建迁移任务。

7.4 证据链抽屉

用于所有 AI 生成结果：

```
——— 证据链 ———
生成对象：TP-PFC-GEN3-0001
芯片代系：Switch_Gen3_RevB

[寄存器] PFC_DEADLOCK_TIMER
[字段] THRESHOLD
[SDK] pfc_recovery_enable(...)
[Bug] BUG-PFC-2026-0017
[知识块] chunk_id=809

[打开源文件] [查看引用片段]
```

8. 端到端流程设计

8.1 新芯片导入流程

```
创建芯片代系 Switch_Gen3_RevB
→ 上传 SVD XML (file_category=svd, chip_version=Switch_Gen3_RevB)
→ SVDParserService 解析寄存器/字段
→ 上传 SDK headers/source (file_category=sdk_header/sdk_source)
→ SDKAstService 建 API 索引
→ 上传 design/test_plan/environment/errata
```

- fileParserService + chunkingService 分块
- embeddingAdapter 生成 embedding
- 知识库显示：该芯片资产准备完成

8.2 基于知识库生成用例

- 用户选择模块 + 芯片代系 + 知识文件
- caseGeneratorService 创建任务
- ChipContextService 注入 chip_version 上下文
- hybridSearch 检索 design/register/sdk/bug/methodology
- AI 生成 level1_points/test_cases
- 写 evidence_links
- temp 表预览
- 用户确认合并正式库

8.3 自动生成 TCL 脚本

- 用户点击测试点“生成TCL”
- /api/tcl-generation/generate
- tclGenerationService.generate
- 检索 cli/tcl_convention/register_map/register_field/bug_rag/execution_experience
- 生成三段式 TCL
- _staticValidation 静态校验
- scriptRunnerClient 调 SCRIPT_RUNNER_URL 执行
- 失败则 _selfRepairLoop 修复
- 保存 .tcl 到 uploads/scripts
- 写 test_case_scripts / tcl_generation_tasks / evidence_links
- 执行失败经验进入 bug_rag_entries(candidate)

8.4 从 Gen2 迁移到 Gen3_RevB

- 选择源芯片 Switch_Gen2 与目标芯片 Switch_Gen3_RevB
- SVD diff 找出地址/字段变化
- SDK diff 找出 API 变化
- 测试点继承评估
- 旧脚本扫描：识别硬编码地址、旧 API、旧 CLI
- Agent 生成迁移建议
- 用户确认
- 批量生成新测试点/新脚本
- 执行环境验证
- 结果进入正式库

9. Prompt 与 Agent 规则设计

9.1 Planner Prompt 要点

Planner 负责决定如何把稳定测试逻辑映射到当前芯片事实。

必须输入：

- `chip_version`
- 继承链
- 模块名
- 选中文档摘要

- 寄存器差异
- SDK API 差异
- Bug-RAG 命中

必须输出：

```
{
  "plan_id": "TP-PFC-GEN3-0001",
  "target_chip": "Switch_Gen3_RevB",
  "reused_logic": ["PFC recovery flow from Gen2"],
  "chip_specific_facts": ["PFC_DEADLOCK_TIMER.THRESHOLD"],
  "risk_items": ["POLL_TIMEOUT", "LEGACY_ADDR_HARDCODED"],
  "test_points": []
}
```

9.2 Scripter Prompt 要点

必须遵循：

1. 不能编造 CLI 命令。
2. 不能硬编码旧代寄存器地址。
3. 所有寄存器字段必须引用 `register_map` / `register_field` 证据。
4. 必须包含 `setup`、`test_steps`、`teardown`。
5. 必须使用 `catch` 包裹关键操作。
6. 必须输出证据注释。
7. 如果证据不足，输出 `needs_human`，而不是编造脚本。

9.3 Critic/Lint 规则

规则	检查内容	失败处理
结构规则	是否包含 setup/test_steps/teardown/catch	自动修复
语法规则	括号、方括号、proc 定义	自动修复
证据规则	CLI/寄存器/SDK 是否有 evidence	needs_human
芯片规则	是否出现旧代地址/旧 API	自动修复或 needs_human
安全规则	是否有危险命令、无清理动作	needs_human
环境规则	是否适配目标环境	自动修复

10. 关键实现步骤与优先级

10.1 P0：先修现有能力闭环（1周）

1. 修复 `/services/fileParserService` 中 TCL 学习后处理不可达问题。
2. 确认 `ai_material_chunks` 已包含 `embedding`、`library_id`、`file_category`、`chunking_strategy` 等字段。
3. 确认 `tcl_generation_tasks`、`tcl_learning_results` 已迁移。
4. 为 `tclGenerationService._assembleKnowledge` 增加 `execution_experience` 检索。
5. 在 UI 上确认 `generateTCL`、`expandPoints` 能完整显示任务状态。

6. 不启用 `EMBEDDING_BACKEND=chroma` 。

验收标准：上传 TCL 历史脚本后能自动生成 CLI/TCL 规范；点击测试点能生成 TCL；未配置 runner 时状态为 `skipped` 但任务不失败。

10.2 P1: 芯片代系与证据链（2-3 周）

1. 新增 `chip_versions` 、 `evidence_links` 。
2. `module_knowledge_files` 、 `ai_material_chunks` 、 `level1_points` 、 `test_cases` 、 `tcl_generation_tasks` 增加 `chip_version_id` 。
3. 新增 `ChipContextService` 。
4. 上传 UI 增加芯片代系选择。
5. AI 用例生成与 TCL 生成均写入 evidence。

验收标准：同一模块可按 `Switch_Gen1` / `Switch_Gen2` / `Switch_Gen3` 过滤知识与生成结果；每条 AI 生成用例能查看证据链。

10.3 P2: SVD 与 SDK 结构化入库（3-4 周）

1. 新增 `chip_registers` 、 `chip_register_fields` 。
2. 实现 `SVDParserService` 。
3. 新增 `sdk_api_symbols` 。
4. 实现 `SDKAstService` P1 版本。
5. 扩展 RAG 类别： `register_map` 、 `register_field` 、 `sdk_api` 。

验收标准：上传 SVD 后能查询 `PFC_DEADLOCK_TIMER.THRESHOLD` ；TCL 生成能引用该字段；迁移中心能显示寄存器差异。

10.4 P3: Bug-RAG 与执行环境（3-4 周）

1. 新增 `bug_rag_entries` 。
2. `_writeRepairExperience` 改为先写候选池。
3. 增加人工审核页面。
4. 新增 `execution_environments` 与 `script_execution_runs` 。
5. 扩展 `scriptRunnerClient` 支持不同 runner_url。

验收标准：执行失败能形成 candidate；人工 approve 后进入 `bug_rag` 检索；后续生成能规避同类错误。

10.5 P4: 迁移中心与多 Agent 编排（4-6 周）

1. 新增 `AdaptiveWorkflowService` 。
2. 新增迁移中心页面。
3. 支持 Gen2 → Gen3/RevB 批量迁移。
4. 支持 Planner/Search/Scripter/Critic/Executor/Reflection 全链路任务可视化。
5. 与 `ai_unified_tasks` 深度整合。

验收标准：用户选择源芯片与目标芯片后，系统自动输出继承/重生成/人工确认清单，并能批量生成新脚本。

11. 与现有代码的具体改造清单

11.1 必改文件

文件	改造点
<code>server.js</code>	挂载 <code>/api/chip-versions</code> 、 <code>/api/svd</code> 、 <code>/api/sdk-ast</code> 、 <code>/api/bug-rag</code> 、 <code>/api/adaptive-generation</code>
<code>routes/knowledge.js</code>	上传参数增加 <code>chipVersionId</code> , 扩展 <code>fileCategory</code>
<code>/services/fileParserService</code>	增加 <code>svd/xml</code> 解析分支; 修复 <code>TCL</code> 学习后处理位置
<code>/services/webCrawlerService</code>	增加抓取结果直接绑定 <code>chipVersionId/fileCategory</code> 的保存参数
<code>services/embeddingAdapter.js</code>	增加 <code>chipVersionId</code> 过滤与 <code>hybridSearch</code>
<code>services/caseGeneratorService.js</code>	生成 <code>prompt</code> 注入 <code>chip context</code> ; 写 <code>evidence_links</code>
<code>services/tclGenerationService.js</code>	扩展 <code>RAG</code> 类别; 写 <code>evidence</code> ; 支持 <code>execution env</code>
<code>services/scriptRunnerClient.js</code>	支持按环境选择 <code>runner_url</code> , 而不只读全局 <code>SCRIPT_RUNNER_URL</code>
<code>services/reflectionPipeline.js</code>	增加芯片/寄存器/SDK/证据规则
<code>public/js/level1-points-list.js</code>	<code>TCL</code> 生成弹窗增加芯片代系、执行环境、证据查看
<code>public/css/knowledge-library.css</code>	增加芯片标签、文件类别 <code>badge</code> 样式

11.2 新增文件

```
migrations/20260630_add_chip_adaptive_system.sql
routes/chipVersions.js
routes/svd.js
routes/sdkAst.js
routes/bugRag.js
routes/adaptiveGeneration.js
services/chipContextService.js
services/svdParserService.js
services/sdkAstService.js
services/evidenceService.js
services/bugRagService.js
services/adaptiveWorkflowService.js
services/executionEnvironmentService.js
public/chip-migration.html
public/js/chip-migration.js
public/css/chip-migration.css
public/js/evidence-drawer.js
public/css/evidence-drawer.css
```

12. 风险与规避

风险	影响	规避
ChromaDB 后端未实现	设置 <code>EMBEDDING_BACKEND=chroma</code> 会导致检索失败	P1/P2 坚持 <code>MySQL backend</code> ; <code>Chroma</code> 只作为 P4 可选
<code>TCL</code> 学习后处理不可达	历史脚本无法自动沉淀规范	P0 优先修复 <code>/services/fileParserService</code>
AI 编造寄存器/CLI	生成脚本不可执行或误操作	<code>Critic</code> 规则要求 <code>evidence</code> , 不足则 <code>needs_human</code>
旧代地址硬编码	迁移到新芯片失败	<code>SVD diff</code> + <code>lint</code> 扫描 <code>LEGACY_ADDR_HARDCODED</code>

风险	影响	规避
Runner 未配置	动态执行跳过	UI 明确显示 ScriptRunner Agent未配置 ，允许先静态验证
Bug-RAG 污染	错误经验反向污染生成	所有失败经验先 candidate，经人工审核后入库
文件类型膨胀	知识库混乱	file_category 标准化，UI badge 化
数据迁移破坏现有表	影响生产系统	所有新增字段允许 NULL，新增表不改现有主流程

13. 示例：Gen3 PFC Deadlock Recovery 落地流程

13.1 数据准备

上传：

- Switch_Gen3_RevB.svd → file_category=svd → 解析出 PFC_DEADLOCK_TIMER.THRESHOLD。
- sdk_gen3_dev/include/pfc.h → file_category=sdk_header → 解析 SDK API。
- PFC 测试计划 → file_category=test_plan。
- 历史 TCL 脚本 → file_category=tcl_script → 学习 CLI/TCL 规范。
- 已知 bug：BUG-PFC-2026-0017 → file_category=bug_rag 或写入候选后审核。

13.2 用例生成

输入上下文：

```
{
  "chip_version": "Switch_Gen3_RevB",
  "module": "PFC",
  "test_point": "pfc_recovery_fsm_clear_timeout",
  "registers": ["PFC_DEADLOCK_TIMER", "PFC_RECOVERY_STATUS"],
  "known_bugs": ["BUG-PFC-2026-0017", "POLL_TIMEOUT"]
}
```

输出测试点：

```
TP-PFC-GEN3-0001: PFC deadlock recovery FSM clear timeout 验证
```

输出证据：

```
EB-PFC-GEN3-0001: SVD字段 PFC_DEADLOCK_TIMER.THRESHOLD
REF-PFC-20260630-001: PFC Recovery 测试计划章节
BUG-PFC-2026-0017: 旧版本 POLL_TIMEOUT 规避经验
```

13.3 TCL 生成

生成脚本必须包含：

```
proc setup {} { ... }
proc test_steps {} { ... }
proc teardown {} { ... }
```

并在注释中标出证据：

```
# Evidence:
# - TP: TP-PFC-GEN3-0001
# - Register: PFC_DEADLOCK_TIMER.THRESHOLD
# - Bug-RAG: BUG-PFC-2026-0017 / POLL_TIMEOUT
```

13.4 执行与回灌

```
执行环境: xcelium_sim
→ 失败: POLL_TIMEOUT
→ selfRepairLoop 修复轮询条件
→ 再执行 passed_after_repair
→ 提取经验: 避免固定 sleep, 应轮询 PFC_RECOVERY_STATUS.FSM_STATE == RECOVERED
→ 写入 bug_rag_entries(candidate)
→ 人工审核 approve
→ 后续生成自动规避
```

14. 邮件与报告能力接入建议

当前本次报告使用外部脚本：

```
/Users/zhao/.openclaw/workspace/scripts/send_qq_mail.py
```

该脚本支持：

- Markdown 转精美 HTML。
- `--save-html` 保存 HTML。
- `--attach-pdf` 调 Chrome headless 生成 PDF 附件。
- 默认收件人：`13913141114@163.com`、`zhaosz@centec.com`。
- SMTP：`smtp.qq.com`，SSL 端口 `465`。
- 发件人：`645388352@qq.com`，发件人名 `AI领航员·DC技术研究助手`。

xTest 内部也已有 `services/emailService.js`，但它更偏通知邮件。建议后续把报告能力做成：

```
report_generation 任务
→ Markdown/HTML 报告模板
→ Chrome/LibreOffice PDF 导出
→ emailService 发送
→ email_logs 记录
```

短期仍可使用 `scripts/send_qq_mail.py` 作为稳定交付路径。

15. 最终结论

xTest 已经拥有实现《智能交换芯片多代系自适应自动化测试系统详细设计文档 V2.0》的核心底座，尤其是知识库、AI 生成、TCL 生成、TCL 学习、反思修复和任务中心。真正需要补齐的是：

1. 芯片代系上下文：让 `chip_version` 成为知识、用例、脚本、执行结果的统一维度。

- 2. **SVD 结构化解析**：把寄存器和字段从文档变成可查询、可引用、可校验的事实层资产。
- 3. **SDK API 索引**：把 SDK 函数、宏、结构体变成生成与迁移可用的工程事实。
- 4. **证据链**：让每次 AI 输出都能追溯到知识块、寄存器、SDK API、Bug、执行日志。
- 5. **Bug-RAG 候选审核**：让失败经验沉淀，但不污染知识库。
- 6. **迁移中心**：把 Gen1/Gen2/Gen3/RevB 的继承与差异变成可操作任务。
- 7. **执行环境矩阵**：将 `xcelium_sim`、`vcs_sim`、`fpga_lab`、`silicon_bringup`、`sdk_unit_test` 纳入统一执行视图。

建议以 P0→P4 渐进路线落地，不要一开始引入 ChromaDB/Redis/复杂 Agent 框架。当前 xTest 的工程方向是对的：**MySQL 状态机 + 知识分块 + 轻量 RAG + 人工确认 + 自动化脚本闭环**，只要把芯片代系与证据链补上，就能承接多代智能交换芯片自适应自动化测试系统。

AI领航员·DC技术研究助手

2026-06-30

本报告由 AI领航员 自动生成，仅供研究参考

 反馈直接回复本邮件 ·  配置: ~/.openclaw/workspace/scripts/send_qq_mail.py